

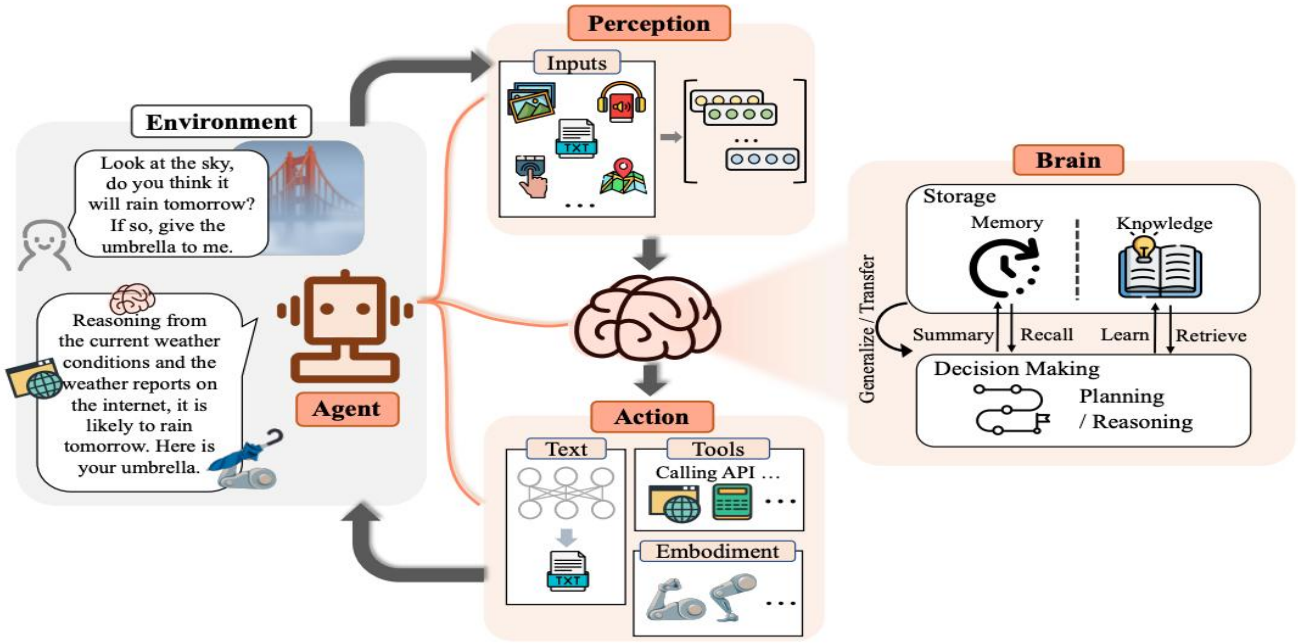
Coding Agent第3次课
Skill 与 Agent 开发

MCP协议 · Skill开发实践 · 协同架构设计

郑海超
西南财经大学 管理科学与工程学院

Agent（代理、智能体）

- 代理是专门的 AI 助手，可以针对特定任务和工作流程进行配置。它们允许您创建具有自定义提示词、模型和工具访问权限的专用工具。
- OpenCode 中有两种类型的代理：主代理和子代理。
  - OpenCode 内置了两个主代理：Build 和 Plan。
- 小组项目可运行的结果也可以是Agent



代理 | OpenCode

Contents

课程目录

渐进式学习路径：从MCP协议理解零代码数据获取的价值，到自主开发金融分析Skill掌握AI辅助编程方法，最终通过Agent架构实现多工具协同的端到端工作流。

01 MCP协议与工具扩展

理解MCP作为Model Context Protocol的核心价值，掌握远程URL型的配置方法

02 Skill开发实践

掌握SKILL.md文件结构，通过股票技术分析Skill案例实践双通道数据获取策略

03 Agent协同架构

理解Agent作为自主调度者的角色，通过金融数据合规检查与日报生成案例实践三种设计模式

PRE-CLASS THINKING

课前思考：从痛点出发

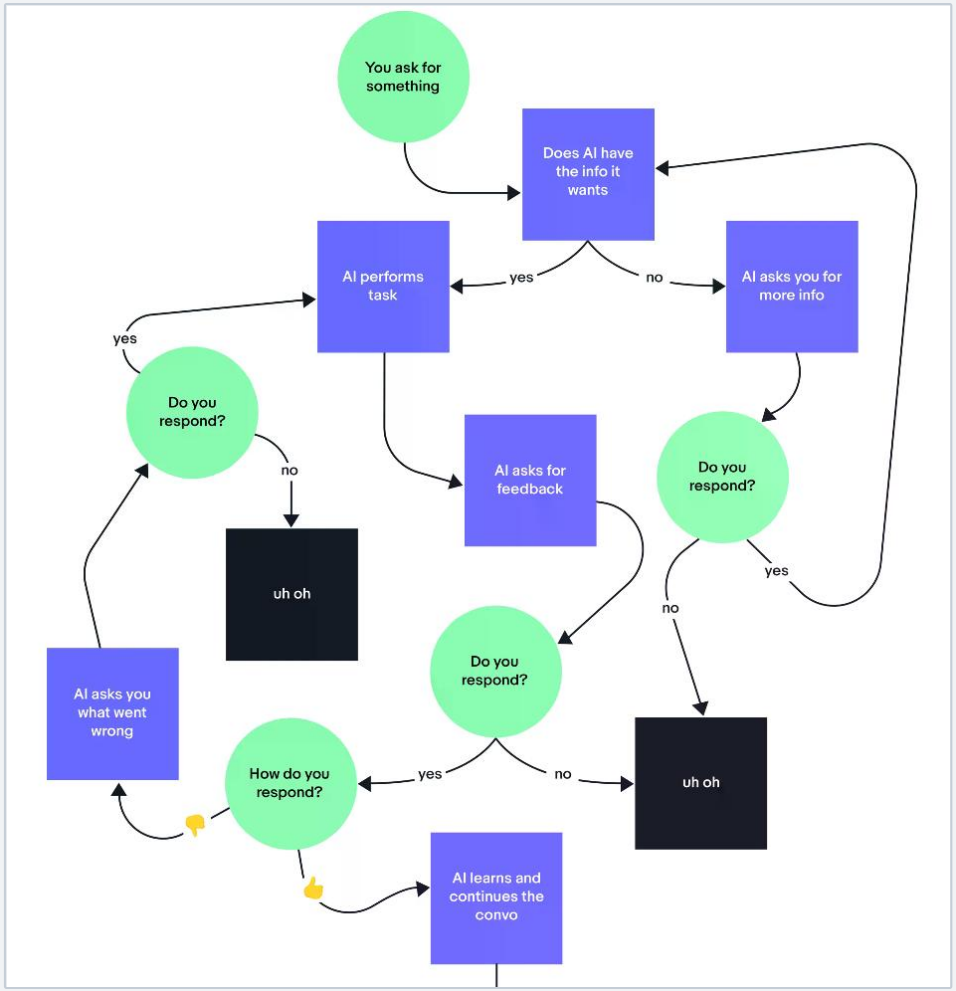
传统Skill方式通过AI生成并执行Python代码来获取数据，存在链路长、延迟高、错误点多的结构性缺陷。本节课通过引入MCP协议实现零代码数据获取，通过Skill开发实现自定义分析逻辑，通过Agent架构实现多工具协同，构建更稳健高效的AI辅助开发工作流。

回顾第2节Coding Agent课的痛点：tushare Skill的数据获取链路为"用户输入→AI读SKILL.md→写Python
代码→运行→解析结果"，五个环节均可能出错，包括语法错误、依赖冲突、网络超时、格式解析异常等

01 **核心问题一：**若AI能直接调用外部工具接口而非通过生成代码间接调用，可大幅缩短数据获取链路，降低延迟与错误概率

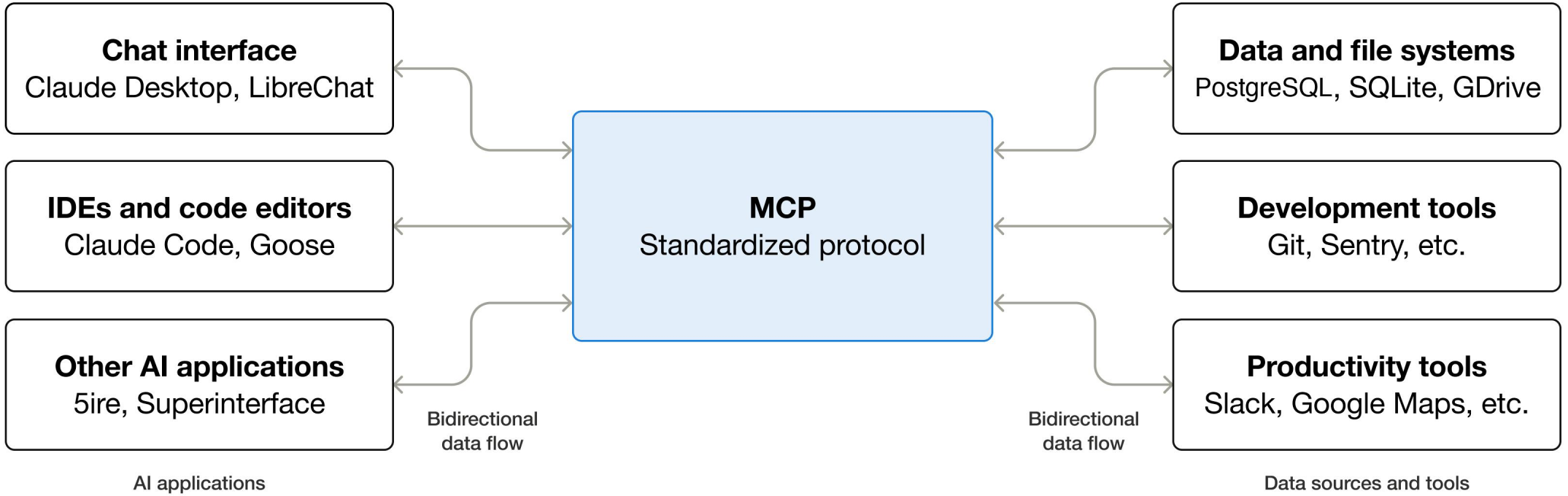
02 **核心问题二：**从使用预设Skill过渡到自主开发Domain-Specific Skill，需要掌握SKILL.md
编写规范、步骤拆解方法与错误处理策略

03 **核心问题三：**单一工具难以完成端到端任务，需通过Agent架构实现MCP（数据获取）、Skill（分析方法）、Agent（任务调度）的三方协同



AI工具链与数据流概念图

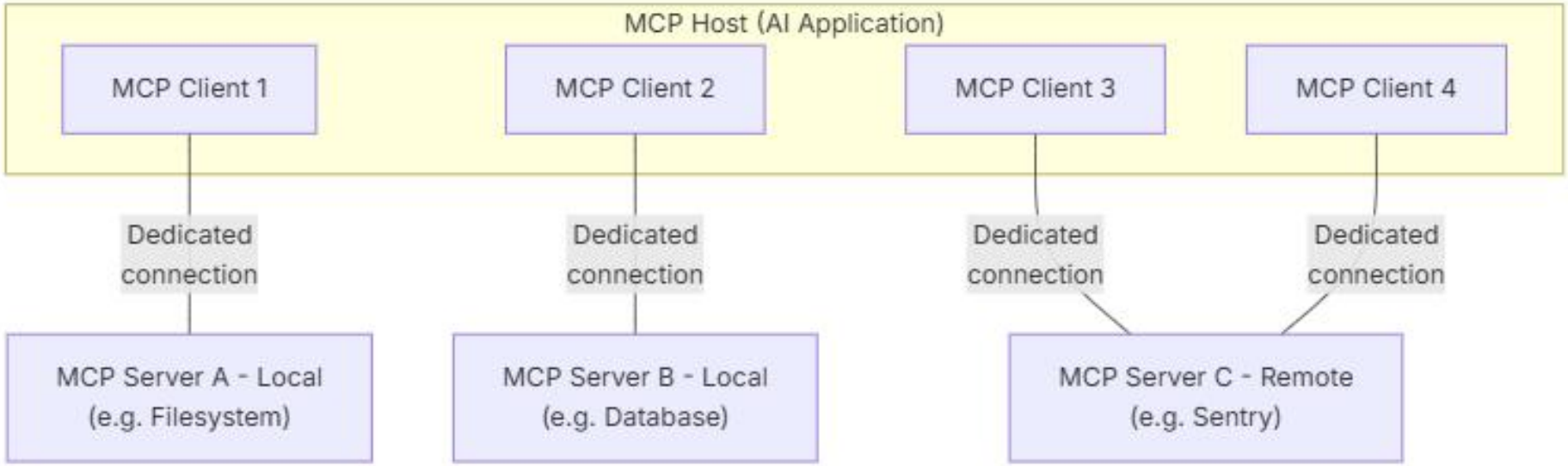
What is the Model **Context** Protocol (MCP)?



https://modelcontextprotocol.io/docs/getting-started/intro

MCP Architecture

- MCP follows a **client-server** architecture where an MCP host — an AI application like Claude Code or opencode — establishes connections to one or more MCP servers.



https://modelcontextprotocol.io/docs/learn/architecture

技术架构对比

MCP：零代码工具调用协议（比较Skill和MCP）

> MCP（Model Context Protocol）通过标准化接口协议让AI直接调用外部工具，将传统Skill的"生成代码-执行-解析"三阶段压缩为"直接请求-返回结果"单阶段，实现真正的零代码数据获取。

- **协议定义：**标准化接口规范，使AI直接调用外部数据源，无需生成中间代码
- **链路对比：**传统方式需"读说明→写代码→运行→解析"四步，MCP仅需两步
- **核心优势：**消除代码生成环节的语法错误与依赖管理开销
- **适用边界：**标准化数据查询场景最优，复杂逻辑仍需Skill支持

Skill vs MCP 核心差异对比

对比维度	Skill 方式	MCP 方式
本质定义	告诉AI"怎么做"的说明书	给AI"直接用的工具"接口
交互流程	AI写代码→运行→解析结果	AI发送请求→直接返回结果
是否需要写代码	是（中间步骤必须）	否（零代码）
延迟性能	较高（代码执行开销）	较低（直接调用）
灵活性程度	高（可自定义任意逻辑）	中（取决于接口能力）

MCP适合标准化数据获取，Skill适合复杂逻辑分析，两者可协同使用

MCP Configuration

MCP 配置方法与最佳实践

MCP配置需在opencode.jsonc中定义服务器类型（remote/local）、连接参数及权限控制。remote类型通过HTTPS端点直接连接云服务，local类型通过stdio启动本地进程。

Configuration

配置位置与格式

在项目根目录创建opencode.jsonc文件，在mcp字段下定义服务器配置，每个服务器需指定type、url或command及环境变量。

Remote Type

Remote远程URL型

通过HTTPS端点直接连接，适用于云API服务（如tushare MCP），配置简单无需本地安装，支持token参数通过env引用。

Local Type

Local本地命令型

通过子进程启动本地stdio服务，适用于本地桌面应用集成（如Zotero MCP），AI通过标准输入输出流与服务通信。

Security

安全规范

严禁硬编码API Token等敏感信息，必须使用{env:VARNAME}占位符语法，配合环境变量或CI/CD密钥管理服务。

Type Comparison

配置类型对比

Remote: 云托管，HTTPS直连
Local: 本地进程，stdio通信

Scope Comparison

配置级别对比

项目级: 根目录opencode.jsonc，团队共享
全局级: ~/.config/，个人跨项目通用

案例一：配置 tushare MCP

通过配置tushare MCP服务器，体验零代码获取金融数据的完整流程

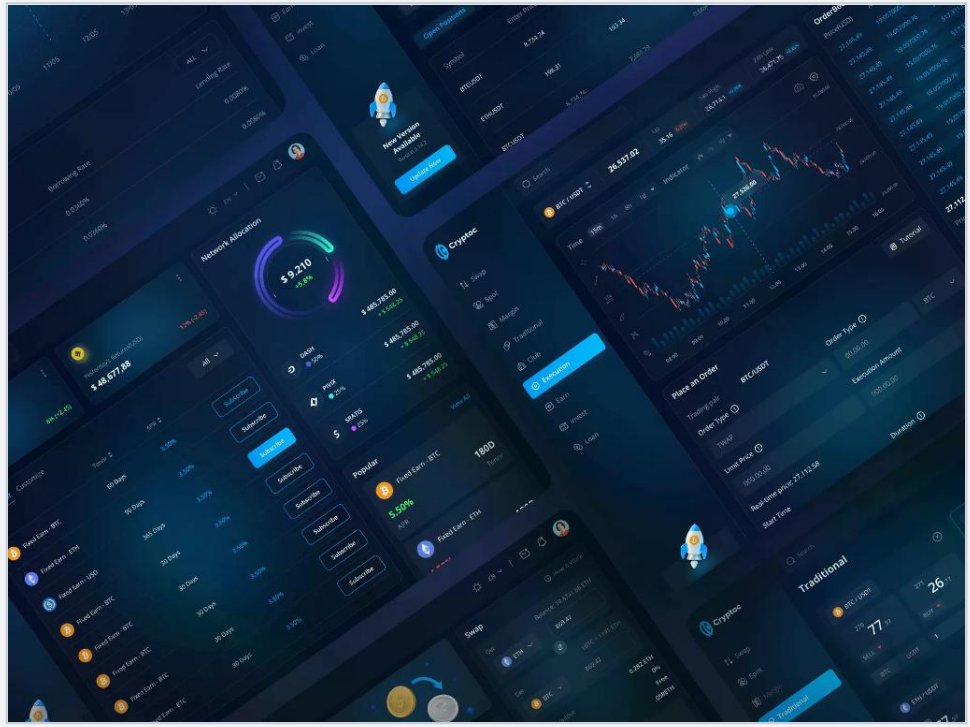
配置完成后，用户通过自然语言指令（如"获取贵州茅台最近30天日线数据"）即可触发AI直接调用MCP接口，无需经历编写Python代码、处理依赖、解析返回等中间环节，直观感受MCP在纯数据获取场景下的效率优势。

01 **配置步骤：**在项目根目录创建opencode.jsonc文件，定义tushareMcp对象，设置type为remote，url指向API端点

02 **服务发现：**重启opencode会话后，AI自动扫描配置文件，将tools注册到当前上下文中

03 **自然语言调用：**输入自然语言指令，AI解析意图后直接构造HTTP请求调用接口

04 **对比验证：**与Skill方式对比，MCP无需生成Python代码，数据获取延迟从秒级降至毫秒级



tushare金融数据平台界面

SKILL.md 编写规范与结构

SKILL.md作为Skill的定义文件，采用Markdown格式组织，包含YAML frontmatter元数据区与结构化正文。Frontmatter定义Skill的标识名称、功能描述与适用场景；正文通过"背景-步骤-示例-注意事项"四层结构，将复杂任务拆解为AI可执行的原子操作。

FRONTMATTER

name字段使用简短有意义的标识符（如stock-tech-analysis），description必须包含适用场景、关键动词与领域关键词，直接影响AI触发Skill的准确性。

四层结构

背景说明阐述数据来源与分析目标；步骤说明将任务拆解为原子操作（数据获取→清洗→计算→可视化→输出），每个步骤明确工具选择。

指令具体化

禁止使用模糊动词如"分析"、"处理"，必须指定具体算法与参数（如"计算20日简单移动平均、MACD（12,26,9）、RSI（14日）"）。

错误处理

在注意事项中列出金融数据常见异常（股票代码无效、停牌日缺失、数据不足、API限频），并指定检测方法与处理策略。

金融 Skill 设计：步骤拆解与错误处理

核心洞察：金融数据分析Skill的设计遵循"场景分析→步骤拆解→指令编写→错误处理→测试验证"五阶段流程。针对股票技术分析场景，任务被拆解为数据获取、数据清洗、指标计算、可视化绘图、报告生成五个原子步骤，并建立金融数据特异的错误处理矩阵。

01 **双通道数据获取策略：**优先调用tushare MCP实现零代码获取，若MCP服务不可用则自动回退至tushare Skill，确保数据获取环节的容错性。

02 **技术指标计算规范：**明确指定pandas计算20日均线，talib计算MACD（12/26/9参数）、RSI（14日周期），避免AI使用默认参数导致结果偏差。

03 **金融数据边界条件处理：**停牌日数据采用前值填充；MACD计算设置前置检查（至少35个交易日）；RSI添加除零保护逻辑（价格无变化时返回中性值50）。

金融数据常见错误与处理策略

错误类型	检测方法	处理策略
股票代码无效	tushare返回空DataFrame	提示用户检查代码格式（如600519.SH）
停牌日数据缺失	交易日期不连续	使用前值填充或标注停牌状态
数据不足计算指标	数据行数<计算周期	提示"MACD至少需要35个交易日"
API限频	返回错误码429	等待后重试或降低请求频率
RSI除零风险	价格变动为0	添加保护逻辑返回中性值50

AI 辅助 Skill 开发：迭代优化流程

AI辅助Skill开发采用"创建→测试→评估→改进"的闭环迭代模式。利用skill-creator工具基于自然语言需求自动生成SKILL.md框架，开发者聚焦填充领域特定细节，随后通过skill-improver进行质量评估，最终实现从需求描述到可用Skill的快速转化。

01

Skill-Creator自动化框架生成

在opencode中输入需求描述（如"创建股票技术分析Skill，计算均线、MACD、RSI并绘图"），AI自动生成符合规范的SKILL.md模板，包含完整的frontmatter、背景说明、步骤占位符、示例结构与注意事项框架，将开发起点从零提升到80%。

02

领域知识填充

基于生成的框架，开发者补充金融特异性细节：在数据获取步骤标注MCP优先/Skill回退策略，在计算步骤明确技术指标参数（均线周期20日、MACD的12/26/9参数），在注意事项中添加金融数据异常处理矩阵。

03

Skill-Improver质量评估

从三个维度检查Skill质量——description准确性（是否能被正确触发）、步骤完整性（每个步骤是否可执行、工具是否明确）、边界覆盖度（是否处理常见异常），输出结构化改进建议。

04

测试驱动迭代

案例二：开发股票技术分析 Skill

本案例通过完整开发股票技术分析Skill，演示从需求到成品的全流程。Skill支持计算20日均线、MACD（12,26,9参数）、RSI（14日）等技术指标并生成可视化图表，其中数据获取环节实现MCP优先、Skill回退的双通道策略，计算环节明确定义pandas与talib工具调用，错误处理环节覆盖停牌缺失填充与数据不足检查，最终通过贵州茅台真实数据验证流程正确性。

> **框架生成与需求输入：**在opencode中调用skill-creator，输入"创建股票技术分析Skill，能计算均线、MACD、RSI指标并绘制图表"，获取包含frontmatter、背景、步骤、示例、注意事项的完整SKILL.md框架

> **技术指标计算指令填充：**在步骤说明中明确指定计算逻辑——20日简单移动平均线使用pandas.DataFrame.rolling(window=20).mean()；MACD使用12日EMA减26日EMA得到DIF，再对DIF计算9日EMA得到DEA；RSI使用14日周期，基于价格变动幅度计算相对强弱

> **双通道数据获取策略标注：**在步骤1明确写入"优先使用tushare MCP获取日线数据（零代码低延迟）；若MCP服务不可用或返回错误，则回退至tushare Skill（编写Python代码通过SDK获取）"，确保数据获取环节的容错性与可靠性

> **质量评估与错误处理完善：**使用skill-improver评估框架，根据建议添加具体错误处理——停牌日数据缺失采用前向填充（ffill）、MACD计算需至少35个交易日数据否则提示错误、RSI计算添加除零保护返回50中性值

> **端到端测试验证：**使用贵州茅台（600519.SH）最近60个交易日数据测试完整流程，验证数据获取成功、技术指标计算数值正确（与通达信/同花顺对比）、图表可视化正常输出，确认Skill达到生产可用标准

TECHNOLOGY

Agent 基础：自主调度者

> Agent作为自主智能体，与被动加载的Skill存在本质差异：Agent拥有独立的Prompt、工具集、权限控制与模型配置，具备自主规划、工具调用、子代理委派与结果返回的完整生命周期。

核心特性

主代理（Primary）

用户直接交互入口，通过Tab键循环切换，拥有完整上下文与全量工具权限，负责任务接收与整体编排。内置Build代理（全工具权限）与Plan代理（规划专用）。

子代理（Subagent）

不直接面向用户，由主代理根据description自动选择与委派，专注单一职责，通过标准化输入输出与主代理通信，实现任务解耦与专业化分工。

生命周期管理

经历创建→配置→触发→执行→返回五个阶段，支持动态加载与卸载，具备自主规划、工具调用、子代理委派与结果返回的完整闭环。

Agent vs Skill 核心区别

对比维度	Agent（智能体）	Skill（技能）
本质定义	独立智能体，有自己的prompt、工具、模 型	指令模板，指导AI如何使用工具
运行方式	独立运行，可自主规划与委派	被动加载，必须由Agent调用
配置内容	prompt + tools + permission + model	SKILL.md（步骤+示例+注意事项）
能力范围	可调用子代理、MCP、Skill	仅提供步骤指导

ⓘ Agent是执行者，Skill是指导手册，两者协同完成复杂任务

CONFIGURATION

Agent 创建与配置方法

支持 Markdown（推荐）与 JSON 两种方式，通过 frontmatter 定义元数据，正文编写 prompt，实现与 Skill 开发体验一致的工作流。

Markdown 配置（推荐）

在 `.opencode/agents/{name}.md` 创建文件，顶部使用 YAML frontmatter 定义元数据（description、mode、tools、permission），正文编写 prompt。与 SKILL.md 风格统一，支持长文本清晰展示，便于 Git 版本控制。

JSON 配置方式

`opencode.jsonc`

在 opencode.jsonc 的 agent 字段集中配置，键名为代理标识。配置集中管理，适合简单代理，但 prompt 较长时可读性下降。

核心字段详解

description：描述代理功能与适用场景，主代理据此决定何时委派。

mode：primary（主代理）/ subagent（子代理）/ all（两者皆可）。

tools：控制可用工具（bash/read/write/edit 等布尔值）。

permission：allow（直接执行）/ ask（需确认）/ deny（禁止）。

交互式创建工具

`opencode agent create`

运行 opencode agent create，系统交互式询问代理名称、模式、描述、工具权限等信息，自动生成配置文件，降低手动编写格式错误风险，适合初学者快速上手。

CASE STUDY 03

案例三：金融数据合规检查代理

创建子代理 data-auditor，检测股票数据中的三类异常：负价格、日期不连续、涨跌幅超限，输出结构化合规报告。对比 Markdown 与 JSON 两种配置方式的差异与优势。

Step 01

功能需求定义

创建子代理 data-auditor，专门负责金融数据质量检查。输入为股票代码或 CSV 文件路径，输出结构化合规报告。需检测三类异常：负价格、日期序列连续性、涨跌停规则合规性。

Step 02

Markdown 方式配置（课堂实验采用这种方式）

在 .opencode/agents/data-auditor.md 创建文件，frontmatter 设置 description、mode 为 subagent、tools 开启 bash 与 read 权限。正文定义检查逻辑与输出格式，使用 Markdown 表格分类展示异常。

Step 03

JSON 方式配置

在 opencode.jsonc 的 agent 字段定义 data-auditor 对象，包含等效的 description、mode、prompt、tools 配置。验证两种配置方式在代理行为上的等效性，确认 AI 解析后的执行逻辑一致。

Step 04

测试与对比验证

使用贵州茅台 CSV 数据测试，分别加载两种配置方式创建的代理，检查输出报告的一致性。讨论 Markdown 优势：与 Skill 开发风格统一、便于 Git 版本追踪、长 prompt 内容可读性更强。

ARCHITECTURE

协同架构：Skill + Agent + MCP

三者构成层次化协作体系：MCP提供标准化数据接口（工具层），Skill封装领域分析方法（方法层），Agent负责任务调度与编排（调度层）

三层架构定位

MCP层： 提供零代码数据接口（如tushare MCP行情接口），解决数据获取问题

Skill层： 封装领域特定分析逻辑（如股票技术分析算法），解决怎么做的问题

Skill驱动模式

适用于**离线数据分析**场景，Agent加载分析Skill，使用内置bash工具执行Python代码进行数据清洗与技术指标计算，不依赖外部MCP服务，适用于历史数据批量处理与复杂自定义算法。

纯代码分析 • 离线处理

MCP增强模式

适用于**实时数据查询**场景，Agent先调用tushare MCP获取最新行情数据（零代码低延迟），再加载分析Skill对获取的数据进行计算与解读，结合两者优势实现高效实时分析。

零代码获取 + 代码分析

全链路协同模式

适用于**复杂端到端任务**（如自动生成日报），主代理接收用户指令后，按序委派数据采集子代理（调用MCP获取多维度数据）→报告生成子代理（基于模板撰写分析文案）→主代理汇总格式化输出，实现多专业分工协作。

多代理流水线 • 端到端自动化

设计模式选择：根据任务复杂度可采用Skill驱动（纯代码分析）、MCP增强（零代码获取+代码分析）、全链路协同（多代理流水线）三种模式，主代理通过description自动识别并委派专业子代理，实现数据采集、分析计算、报告生成的无缝协作。

案例四：股票日报自动生成系统

本案例构建端到端金融分析工作流，演示主代理与子代理的分层协作机制。系统由日报Agent统一接收用户指令并编排任务，向下游委派数据采集与报告生成子代理，实现复杂任务解耦与全链路协同。

角色与职责划分

日报Agent（主代理，primary模式）作为统一入口，负责自然语言意图解析、任务拆解与委派；数据采集子代理专注金融数据获取，配置tushare MCP优先策略；报告生成子代理专注文案撰写，输出Markdown格式报告。

数据流架构设计

用户输入→日报Agent→委派data-collector子代理→调用tushare MCP获取指数行情与板块数据→返回结构化数据→委派report-writer子代理→基于模板撰写行情概览、板块分析、重点关注三部分→主代理汇总格式化→输出完整日报。

配置实现

在.opencode/agents/目录分别创建data-collector.md（mode=subagent，tools=bash，prompt定义使用MCP/Skill获取数据并输出JSON）与report-writer.md（mode=subagent，tools=write，prompt定义根据数据撰写Markdown日报）；在opencode.jsonc配置daily-report主代理。

运行演示

使用Tab键切换到daily-report主代理，输入"生成今日股市日报"，观察主代理如何逐步解析需求、自动选择并委派子代理、传递上下文数据、汇总最终结果，验证三层协同架构在复杂任务中的可行性与效率。

Summary

总结与核心要点

本课程构建了AI辅助开发的完整工具链认知：MCP解决零代码数据获取问题，Skill解决自定义分析逻辑问题，Agent解决多工具协同调度问题。三者通过分层协作形成"工具层-方法层-调度层"架构，根据任务场景灵活选用Skill驱动、MCP增强或全链路协同模式。

01 MCP核心价值与选型

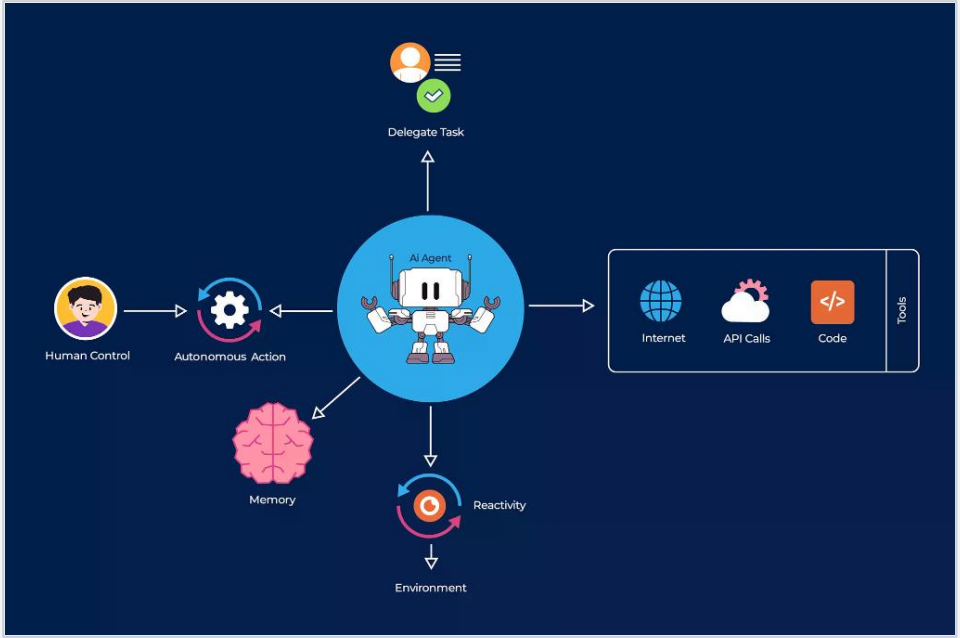
MCP通过标准化接口实现AI直接调用外部工具，消除代码生成与执行环节。选型原则：纯数据获取优先MCP（零代码低延迟），多步骤复杂分析优先Skill，数据获取+复杂分析采用MCP+Skill协同。

02 Skill开发方法论

遵循场景分析→步骤拆解→指令编写→错误处理→测试验证五阶段流程。SKILL.md中description需包含场景与关键词，步骤指令需具体到工具调用级别，错误处理需前置定义金融数据异常的检测与修复策略。

03 Agent协同架构设计

Agent作为自主调度者，与Skill（方法手册）、MCP（工具接口）形成三层协作。设计原则：单一职责（每个Agent只做一件事）、松耦合（Agent间通过数据传递）、渐进增强（先Skill驱动，再MCP增强，最后全链路协同）。



Agent、Skill、MCP三者协作关系

谢谢观看